

Coding & Solution

Date	03 July 2026
Team ID	xxxxxx
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/goswamitithi1005-byte/emotion_learning_assistant.git
Programming Language(s)	Python
Framework(s) Used	Streamlit, Keras (Torch Backend), PyTorch, Hugging Face Transformers, Plotly, Pandas
Key Features Implemented	Dual-Model Inference Pipeline, fine-tuned DistilBERT transformer, In-Memory Config Scrubbing Engine, Keyword Rule Boosting Layers, GenAI Personalized Guidance
Pending / Incomplete Features	None. Core backend, robust serialization handling, and frontend UI dashboards are fully operational and verified.
Setup / Run Instructions	1. Clone the repository and change directory into the root: <pre>git clone https://github.com/tithi-goswami/emotion_learning_assistant.git cd emotion_learning_assistant</pre> 2. Establish and activate a Python virtual environment: <pre>python -m venv venv venv\Scripts\activate (Windows)</pre> 3. Install full framework prerequisites: <pre>pip install -r requirements.txt</pre> 4. Launch the local application instance server: <pre>streamlit run app.py</pre>

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The codebase successfully bridges legacy Keras tokenization formats across system switches via customized pickled unpicklers (_load_pickle_compat) and maps target arrays natively without downstream array classification mismatches (ValueError: y contains unseen labels). All code is fully documented and successfully pushed live to the master version repository branch.